

MLFF-DATA9A7a Material-Profile and Atom-Group Contract Specification

Compositional material identity without material-specific feature ownership

mdstats project

2026-07-30

Contents

1	Scope	1
2	Motivation	1
3	Record families	2
3.1	MaterialProfileProviderIdentity	2
3.2	PhaseComponentIdentity	2
3.3	MaterialProfileIdentity	2
3.4	Atom-group contracts	2
3.5	Condition axes	3
3.6	Independence axes	3
3.7	Aggregate contract	3
4	Provider protocol	3
5	DATA4 integration and compatibility	4
6	Serialization and identity	4
7	Failure semantics	4
8	Acceptance tests	4
9	Deferred work	5

1 Scope

MLFF-DATA9A7a introduces the immutable declarative contracts that identify what kind of material a dataset represents, which atom groups are scientifically meaningful, which condition axes must be covered, and which independent realizations may support statistical evidence.

This stage does **not** calculate RDFs, coordination numbers, angles, local packing, orientational order, interface profiles, rings, cages, or MACE descriptors. Those numerical providers are owned by later DATA9A7 stages or by the authoritative analysis branches. DATA9A7a owns identity and composition only.

2 Motivation

A single flat material enum cannot represent a solid-liquid interface, a porous crystal containing a liquid, or a reactive molecular phase. The profile must be compositional:

material profile
= one or more phase components
+ one geometry
+ chemistry modifiers
+ optional structural extensions
+ atom-group definitions
+ condition axes
+ independence axes

The user SHALL explicitly declare the production profile. Automatic inference may later provide advice, but an inferred suggestion SHALL NOT become identity evidence without user confirmation.

3 Record families

3.1 MaterialProfileProviderIdentity

Identifies the provider and exact configuration that produced declarative profile records. It contains:

- provider ID;
- provider version;
- canonical configuration SHA-256 digest.

It does not certify any physical feature calculation.

3.2 PhaseComponentIdentity

Each phase component records:

- a stable phase ID;
- a phase kind;
- one or more atom-group IDs assigning atoms to that phase;
- optional chemistry modifiers;
- notes.

The built-in phase kinds are:

- crystalline_solid;
- amorphous_solid;
- liquid;
- molecular_or_gas;
- other.

3.3 MaterialProfileIdentity

The material profile records:

- profile ID and version;
- one or more phase components;
- geometry;
- global chemistry modifiers;
- optional structural extensions;
- optional provider identity;
- explicit user-declaration status.

The built-in geometries are `bulk`, `surface`, `interface`, `confined`, `cluster`, and `other`. An interface SHALL contain at least two phase components. An LTA extension SHALL require `zeolite`, and `zeolite` SHALL require `porous_network`. The generic default activates no structural extension.

3.4 Atom-group contracts

`AtomGroupDefinition` assigns a stable scientific group ID to a selector. The selector kinds are:

- all atoms;
- atomic numbers;
- explicit atom indices;
- metadata values;
- a versioned provider;
- a composite set operation over existing groups.

Atom groups may be static for the topology or dynamic per frame. Provider-based membership SHALL be dynamic because spatial regions, interface zones, and state-dependent classifications may change with time. Composite dependencies SHALL reference known groups and form an acyclic graph.

An `AtomGroupCatalog` binds the groups to exactly one material profile and its phase IDs. Overlap is allowed unless a later provider or policy declares a partition requirement. The catalog does not silently classify interface phases. A one-phase profile may use the safe `all_atoms` fallback; a multi-phase profile requires explicit membership.

3.5 Condition axes

A `ConditionAxisDefinition` identifies a condition whose coverage may matter. The value kinds are categorical, continuous, integer, and boolean. Axis roles are:

- coverage;
- stratification;
- challenge;
- reporting.

The baseline catalog declares composition, temperature, pressure, and regime. These are identities only; observed values and coverage decisions remain owned by DATA3-DATA5 records.

3.6 Independence axes

An `IndependenceAxisDefinition` describes a source of statistical independence, for example:

- trajectory run;
- initial configuration;
- structural realization;
- replica;
- thermodynamic seed.

The definition records scope, roles that require the axis, the minimum number of distinct values, and whether it is a leakage barrier. Declaring an axis does not prove that independent realizations exist. DATA5 must still evaluate the actual evidence and assign an independence grade.

3.7 Aggregate contract

`MaterialProfileContracts` binds:

- one `MaterialProfileIdentity`;
- one `AtomGroupCatalog`;
- one `ConditionAxisCatalog`;
- one `IndependenceAxisCatalog`.

Every child catalog SHALL carry the parent profile digest. Every phase group referenced by the profile SHALL exist in the atom-group catalog.

4 Provider protocol

DATA9A7a exposes a runtime-checkable `SystemProfileProvider` protocol with four methods:

```
class SystemProfileProvider(Protocol):
    provider_id: str
    provider_version: str
```

```

def build_profile(self) -> MaterialProfileIdentity: ...
def build_atom_groups(self, profile) -> AtomGroupCatalog: ...
def build_condition_axes(self, profile) -> ConditionAxisCatalog: ...
def build_independence_axes(self, profile) -> IndependenceAxisCatalog: ...

```

The protocol is intentionally limited to declarative contracts. Structural feature, event, and selection-evidence providers are added in DATA9A7b and later stages rather than expanding this contract ambiguously.

5 DATA4 integration and compatibility

DATA4 schema v2 allows `Data4FeatureBundle` to carry `material_profile_contracts`. The field is optional only for compatibility with evidence created before DATA9A7a. New production evidence SHALL supply it once the full profile migration is active.

The DATA4 cache writes a separate `material_profile_contracts.json` artifact when the aggregate is present. Existing DATA4 v1 bundles remain readable and retain their historical digest verification. Supplying a generic material profile SHALL NOT activate LTA feature construction; LTA continues to require an explicit LTA policy until DATA9A7d migrates it behind the extension provider.

6 Serialization and identity

Every record SHALL:

- use a versioned schema;
- normalize ordering deterministically;
- reject duplicate or malformed IDs;
- reject nonfinite or unsupported metadata;
- carry a canonical content digest;
- reject tampered serialized payloads.

Filesystem paths are not part of these profile identities.

7 Failure semantics

The implementation SHALL fail closed for:

- an interface with fewer than two phases;
- missing phase atom groups;
- cross-profile child catalogs;
- unknown or cyclic composite groups;
- provider groups declared static;
- inconsistent numeric and categorical axis constraints;
- invalid extension hierarchy;
- advisory or inferred profiles presented as production identity evidence.

8 Acceptance tests

DATA9A7a is complete when focused tests demonstrate:

1. generic one-phase profile construction without importing or activating LTA;
2. explicit multi-phase interface contracts;
3. atom-number, index, metadata, provider, and composite selector validation;
4. composite dependency-cycle rejection;
5. condition and independence-axis round trips;
6. provider protocol materialization;
7. aggregate cross-profile rejection and tamper detection;

8. DATA4 schema-v2 threading and cache persistence;
9. DATA4 schema-v1 compatibility;
10. dependency-graph and documentation consistency.

9 Deferred work

DATA9A7a does not implement:

- universal structural feature providers;
- phase-specific default feature activation;
- interface-region calculation;
- migration of LTA fields out of DATA4-DATA7;
- material-profile-aware observable comparison;
- production corpus regeneration.

Those responsibilities remain assigned to DATA9A7b-DATA9A8.